第6章 访问权限控制

访问控制（或隐藏具体实现）与“最初的实现并不恰当”有关。

所有优秀的作者，包括那些编写软件的程序员，都清楚其著作的某些部分直至重新创作的时候才变得完美，有时甚至要反复重写多次。如果你把一个代码段放到了某个位置，等过一会儿回头再看时，有可能会发现有更好的方式去实现相同的功能。这正是重构的原动力之一，重构即重写代码，以使得它更可读、更易理解，并因此而更具可维护性[⊖]。

但是，在这种修改和完善代码的愿望之下，也存在着巨大的压力。通常总是会有一些消费者（客户端程序员）需要你的代码在某些方面保持不变。因此你想改变代码，而他们却想让代码保持不变。由此而产生了在面向对象设计中需要考虑的一个基本问题：“如何把变动的事物与保持不变的事物区分开来”。

这对类库（library）而言尤为重要。该类库的消费者必须依赖他所使用的那部分类库，并且能够知道如果类库出现了新版本，他们并不需要改写代码。从另一个方面来说，类库的开发者必须有权限进行修改和改进，并确保客户代码不会因为这些改动而受到影响。

这一目标可以通过约定来达到。例如，类库开发者必须同意在改动类库中的类时不得删除任何现有方法，因为那样会破坏客户端程序员的代码。但是，与之相反的情况会更加棘手。在有域（即数据成员）存在的情况下，类库开发者要怎样才能知道究竟都有哪些域已经被客户端程序员所调用了呢？这对于方法仅为类的实现的一部分，因此并不想让客户端程序员直接使用的情况来说同样如此。如果程序开发者想要移除旧的实现而要添加新的实现时，结果将会怎样呢？改动任何一个成员都有可能破坏客户端程序员的代码。于是类库开发者会手脚被缚，无法对任何事物进行改动。

为了解决这一问题，Java提供了访问权限修饰词，以供类库开发人员向客户端程序员指明哪些是可用的，哪些是不可用的。访问权限控制的等级，从最大权限到最小权限依次为：**public**、**protected**、包访问权限（没有关键词）和**private**。根据前述内容，读者可能会认为，作为一名类库设计员，你会尽可能将一切方法都定为**private**，而仅向客户端程序员公开你愿意让他们使用的方法。这样做是完全正确的，尽管对于那些经常使用别的语言（特别是C语言）编写程序并在访问事物时不受任何限制的人而言，这与他们的直觉相违背。到了本章末，读者将会信服Java的访问权限控制的价值。

不过，构件类库的概念以及对于谁有权取用该类库构件的控制问题都还是不完善的。其中仍旧存在着如何将构件捆绑到一个内聚的类库单元中的问题。对于这一点，Java用关键字**package**加以控制，而访问权限修饰词会因类是存在于一个相同的包，还是存在于一个单独的包而受到影响。为此，要开始学习本章，首先要学习如何将类库构件置于包中，然后就会理解访问权限修饰词的全部含义。

⊖ 请查看Refactoring: Improving the Design of Existing Code，作者Martin Fowler等(Addison-Wesley,1999)。偶尔会有某些人对重构抱有异议，他们认为这些代码工作得极好，重构它们无异于浪费时间。这种思维方式的问题在于对于项目所需的时间和资金来说，最大的部分并非投入到了最初的代码编写上，而是投入到了代码的维护上。因此，使代码更加易于理解就意味着节省了大量的金钱。

6.1 包：库单元

包内包含有一组类，它们在单一的名字空间之下被组织在了一起。

例如，在Java的标准发布中有一个工具库，它被组织在**java.util**名字空间之下。**java.util**中有一个叫做**ArrayList**的类，使用**ArrayList**的一种方式是用其全名**java.util.ArrayList**来指定。

```
//: access/FullQualification.java

public class FullQualification {
  public static void main(String[] args) {
    java.util.ArrayList list = new java.util.ArrayList();
  }
} ///:~
```

210

这立刻就使程序变得很冗长了，因此你可能想转而使用**import**关键字。如果你想要导入单个的类，可以在**import**语句中命名该类：

```
//: access/SingleImport.java
import java.util.ArrayList;

public class SingleImport {
  public static void main(String[] args) {
    ArrayList list = new java.util.ArrayList();
  }
} ///:~
```

现在，就可以不用限定地使用**ArrayList**了。但是，这样做**java.util**中的其他类仍旧是都不可用的。要想导入其中所有的类，只需要使用“*”，就像在本书剩余部分的示例中所看到的那样：

```
import java.util.*;
```

我们之所以要导入，就是要提供一个管理名字空间的机制。所有类成员的名称都是彼此隔离的。**A**类中的方法**f()**与**B**类中具有相同特征标记（参数列表）的方法f()不会彼此冲突。但是如果类名称相互冲突又该怎么办呢？假设你编写了一个**Stack**类并安装到了一台机器上，而该机器上已经有了一个别人编写的**Stack**类，我们该如何解决呢？由于名字之间的潜在冲突，在Java中对名称空间进行完全控制并为每个类创建唯一的标识符组合就成为了非常重要的事情。

到目前为止，书中大多数示例都存于单一文件之中，并专为本地使用（local use）而设计，因而尚未受到包名的干扰。这些示例实际上已经位于包中了：即*未命名包*，或称为*默认包*。这当然也是一种选择，而且为了简单起见，在本书其他部分都尽可能地使用了此方法。不过如果你正在准备编写对在同一台机器上共存的其他Java程序友好的类库或程序的话，就需要考虑如何防止类名称之间的冲突问题。

当编写一个Java源代码文件时，此文件通常被称为编译单元（有时也被称为转译单元）。每个编译单元都必须有一个后缀名**.java**，而在编译单元内则可以有一个**public**类，该类的名称必须与文件的名称相同（包括大小写，但不包括文件的后缀名**.java**）。每个编译单元只能有一个**public**类，否则编译器就不会接受。如果在该编译单元之中还有额外的类的话，那么在包之外的世界是无法看见这些类的，这是因为它们不是**public**类，而且它们主要用来为主**public**类提供支持。

211

6.1.1 代码组织

当编译一个**.java**文件时，在**.java**文件中的每个类都会有一个输出文件，而该输出文件的名称与**.java**文件中每个类的名称相同，只是多了一个后缀名**.class**。因此，在编译少量**.java**文件之后，会得到大量的**.class**文件。如果用编译型语言编写过程序，那么对于编译器产生一个中间文件（通常是一个obj文件），然后再与通过链接器（用以创建一个可执行文件）或类库产生器

(librarian，用以创建一个类库）产生的其他同类文件捆绑在一起的情况，可能早已司空见惯。但这并不是Java的工作方式。Java可运行程序是一组可以打包并压缩为一个Java 文档文件(JAR，使用Java的 **jar**文档生成器）的**.class**文件。Java解释器负责这些文件的查找、装载和解释[⊖]。

类库实际上是一组类文件。其中每个文件都有一个**public**类，以及任意数量的非**public**类。因此每个文件都有一个构件。如果希望这些构件（每一个都有它们自己的独立的**.java**和**.class**文件）从属于同一个群组，就可以使用关键字**package**。

如果使用**package**语句，它必须是文件中除注释以外的第一句程序代码。在文件起始处写：

```
package access;
```

就表示你在声明该编译单元是名为**access**的类库的一部分。或者换种说法，你正在声明该编译单元中的**public**类名称是位于**access**名称的保护伞下。任何想要使用该名称的人都必须使用前面给出的选择，指定全名或者与**access**结合使用关键字**import**。（请注意，Java 包的命名规则全部使用小写字母，包括中间的字也是如此。）

212

例如，假设文件的名称是**MyClass.java**，这就意味着在该文件中有且只有一个**public**类，该类的名称必须是**MyClass**（注意大小写）：

```
//: access/mypackage/MyClass.java
package access.mypackage;

public class MyClass {
  // ...
} ///:~
```

现在，如果有人想用**MyClass**或者是**access**中的任何其他**public**类，就必须使用关键字**import**来使**access**中的名称可用。另一个选择是给出完整的名称：

```
//: access/QualifiedMyClass.java

public class QualifiedMyClass {
  public static void main(String[] args) {
    access.mypackage.MyClass m =
      new access.mypackage.MyClass();
  }
} ///:~
```

关键字**import**可使之更加简洁：

```
//: access/ImportedMyClass.java
import access.mypackage.*;

public class ImportedMyClass {
  public static void main(String[] args) {
    MyClass m = new MyClass();
  }
} ///:~
```

身为一名类库设计员，很有必要牢记：**package** 和**import**关键字允许你做的，是将单一的全局名字空间分割开，使得无论多少人使用Internet以及Java开始编写类，都不会出现名称冲突问题。

6.1.2 创建独一无二的包名

读者也许会发现，既然一个包从未真正将被打包的东西包装成单一的文件，并且一个包可以由许多**.class**文件构成，那么情况就有点复杂了。为了避免这种情况的发生，一种合乎逻辑的做法就是将特定包的所有**.class**文件都置于一个目录下。也就是说，利用操作系统的层次化的文

213

⊖ Java中并不强求必须要使用解释器。因为存在用来生成一个单一的可执行文件的本地代码Java编译器。

件结构来解决这一问题。这是Java解决混乱问题的一种方式，读者还会在我们介绍**jar**工具的时候看到另一种方式。

将所有的文件收入一个子目录还可以解决另外两个问题：怎样创建独一无二的名称以及怎样查找有可能隐藏于目录结构中某处的类。这些任务是通过将**.class**文件所在的路径位置编码成**package**的名称来实现的。按照惯例，**package**名称的第一部分是类的创建者的反顺序的Internet域名。如果你遵照惯例，Internet域名应是独一无二的，因此你的**package**名称也将是独一无二的，也就不会出现名称冲突的问题了（也就是说，只有在你将自己的域名给了别人，而他又以你曾经使用过的路径名称来编写Java程序代码时，才会出现冲突）。当然，如果你没有自己的域名，你就得构造一组不大可能与他人重复的组合（例如你的姓名），来创立独一无二的**package**名称。如果你打算发布你的Java程序代码，稍微花点力气去取得一个域名，还是很有必要的。

此技巧的第二部分是把package名称分解为你机器上的一个目录。所以当Java程序运行并且需要加载.class文件的时候，它就可以确定.class文件在目录上所处的位置。

Java解释器的运行过程如下：首先，找出环境变量CLASSPATH[⊖]（可以通过操作系统来设置，有时也可通过安装程序－用来在你的机器上安装Java 或基于Java的工具－来设置）。CLASSPATH包含一个或多个目录，用作查找.class文件的根目录。从根目录开始，解释器获取包的名称并将每个句点替换成反斜杠，以从CLASSPATH 根中产生一个路径名称（于是，package foo.bar.baz 就变成为 foo\bar\baz或 foo/bar/baz 或其他，这一切取决于操作系统）。得到的路径会与CLASSPATH中的各个不同的项相连接，解释器就在这些目录中查找与你所要创建的类名称相关的.class文件。（解释器还会去查找某些涉及Java解释器所在位置的标准目录。）

214

为了理解这一点，以我的域名**MindView.net**为例。把它的顺序倒过来，并且将其全部转换为小写，**net.mindview**就成了我所创建的类的独一无二的全局名称。（com、edu、org等扩展名先前在Java包中都是大写的，但在Java2中一切都已改观，包的整个名称全都变成了小写。）若我决定再创建一个名为**simple**的类库，我可以将该名称进一步细分，于是我可以得到一个包的名称如下：

```
package net.mindview.simple;
```

现在，这个包名称就可以用作下面两个文件的名字空间保护伞了：

```
//: net/mindview/simple/Vector.java
// Creating a package.
package net.mindview.simple;

public class Vector {
  public Vector() {
    System.out.println("net.mindview.simple.Vector");
  }
} ///:~
```

如前所述，**package**语句必须是文件中的第一行非注释程序代码。第二个文件看起来也极其相似：

```
//: net/mindview/simple/List.java
// Creating a package.
package net.mindview.simple;

public class List {
  public List() {
    System.out.println("net.mindview.simple.List");
  }
} ///:~
```

⊖ 当提及环境变量时，将用到大写字母(CLASSPATH)。

这两个文件均被置于我的系统的子目录下：

```
C:\DOC\JavaT\net\mindview\simple
```

（注意，在本书的每一个文件中的第一行注释都指定了该文件在源代码目录树中的位置，这个信息将由针对本书的自动代码抽取工具使用。）

215

如果沿此路径往回看，可以看到包的名称**com.bruceeckel.simple**，但此路径的第一部分怎样办呢？它将由环境变量CLASSPATH关照，在我的机器上是：

```
CLASSPATH=.;D:\JAVA\LIB;C:\DOC\JavaT
```

可以看到，CLASSPATH可以包含多个可供选择的查询路径。

但在使用JAR文件时会有一点变化。必须在类路径中将JAR文件的实际名称写清楚，而不仅是指明它所在位置的目录。因此，对于一个名为**grape.jar**的JAR文件，类路径应包括：

```
CLASSPATH=.;D:\JAVA\LIB;C:\flavors\grape.jar
```

一旦类路径得以正确建立，下面的文件就可以放于任何目录之下：

```
//: access/LibTest.java
// Uses the library.
import net.mindview.simple.*;

public class LibTest {
  public static void main(String[] args) {
    Vector v = new Vector();
    List l = new List();
  }
} /* Output:
net.mindview.simple.Vector
net.mindview.simple.List
*///:~
```

当编译器碰到**simple**库的**import**语句时，就开始在CLASSPATH所指定的目录中查找，查找子目录**net\mindview\simple**，然后从已编译的文件中找出名称相符者（对**Vector**而言是**Vector.class**，对**List**而言是**List.class**）。请注意，**Vector**和**List**中的类以及要使用的方法都必须是public的。

对于使用Java的新手而言，设立CLASSPATH是很麻烦的一件事（我最初使用时就是这样的）；为此，Sun将Java2中的JDK改造得更聪明了一些。在安装后你会发现，即使你未设立CLASSPATH，你也可以编译并运行基本的Java程序。然而，要编译和运行本书的源码包（从www.MindView.net网站可以取得），就得向你的CLASSPATH中添加本书程序代码树中的基目录了。

216

练习1：(1) 在某个包中创建一个类，在这个类所处的包的外部创建该类的一个实例。

冲突

如果将两个含有相同名称的类库以“*”形式同时导入，将会出现什么情况呢？例如，假设某程序这样写：

```
import net.mindview.simple.*;
import java.util.*;
```

由于**java.util.***也含有一个**Vector**类，这就存在潜在的冲突。但是只要你不写那些导致冲突的程序代码，就不会有什么问题—这样很好，否则就得做很多的类型检查工作来防止那些根本不会出现的冲突。

如果现在要创建一个**Vector**类的话，就会产生冲突：

```
Vector v = new Vector();
```

这行到底取用的是哪个**Vector**类？编译器不知道，读者同样也不知道。于是编译器提出错

误信息，强制你明确指明。举例说明，如果想要一个标准的Java **Vector**类，就得这样写：

```
java.util.Vector v = new java.util.Vector();
```

由于这样可以完全指明该**Vector**类的位置（配合CLASSPATH），所以除非还要使用java.util中的其他东西，否则就没有必要写**import java.util.***语句了。

或者，可以使用单个类导入的形式来防止冲突，只要你在同一个程序中没有使用有冲突的名字（在使用了有冲突名字的情况下，必须返回到指定全名的方式）。

练习2：(1) 将本节中的代码片段改写为完整的程序，并校验实际所发生的冲突。

6.1.3 定制工具库

具备了这些知识以后，现在就可以创建自己的工具库来减少或消除重复的程序代码了。例如，我们已经用到的**System.out.println()**的别名可以减少输入负担，这种机制可以用于名为Print的类中，这样，我们在使用该类时可以用一个更具可读性的静态**import**语句来导入：

217

```
//: net/mindview/util/Print.java
// Print methods that can be used without
// qualifiers, using Java SE5 static imports:
package net.mindview.util;
import java.io.*;

public class Print {
  // Print with a newline:
  public static void print(Object obj) {
    System.out.println(obj);
  }
  // Print a newline by itself:
  public static void print() {
    System.out.println();
  }
  // Print with no line break:
  public static void printnb(Object obj) {
    System.out.print(obj);
  }
  // The new Java SE5 printf() (from C):
  public static PrintStream
  printf(String format, Object... args) {
    return System.out.printf(format, args);
  }
} ///:~
```

可以使用打印便捷工具来打印String，无论是需要换行（**print()**）还是不需要换行（**printnb()**）。

可以猜到，这个文件的位置一定是在某个以一个CLASSPATH位置开始，然后接着是net/mindview的目录下。编译完之后，就可以用**import static**语句在你的系统上使用静态的**print()**和**printnb()**方法了。

```
//: access/PrintTest.java
// Uses the static printing methods in Print.java.
import static net.mindview.util.Print.*;

public class PrintTest {
  public static void main(String[] args) {
    print("Available from now on!");
    print(100);
    print(100L);
    print(3.14159);
  }
} /* Output:
Available from now on!
100
```

218

```
100
3.14159
*///:~
```

这个类库的第二个构件可以是在第4章中引入的**range()**方法，它使得foreach语法可以用于简单的整数序列：

```
//: net/mindview/util/Range.java
// Array creation methods that can be used without
// qualifiers, using Java SE5 static imports:
package net.mindview.util;

public class Range {
  // Produce a sequence [0..n)
  public static int[] range(int n) {
    int[] result = new int[n];
    for(int i = 0; i < n; i++)
      result[i] = i;
    return result;
  }
  // Produce a sequence [start..end)
  public static int[] range(int start, int end) {
    int sz = end - start;
    int[] result = new int[sz];
    for(int i = 0; i < sz; i++)
      result[i] = start + i;
    return result;
  }
  // Produce a sequence [start..end) incrementing by step
  public static int[] range(int start, int end, int step) {
    int sz = (end - start)/step;
    int[] result = new int[sz];
    for(int i = 0; i < sz; i++)
      result[i] = start + (i * step);
    return result;
  }
} ///:~
```

219

从现在开始，你无论何时创建了有用的新工具，都可以将其添加到你自己的类库中。你将看到在本书中还有更多的构件添加到了**net.mindview.util**类库中。

6.1.4 用 import改变行为

Java没有C的条件编译功能，该功能可以使你不必更改任何程序代码，就能够切换开关并产生不同的行为。Java去掉此功能的原因可能是因为C在绝大多数情况下是用此功能来解决跨平台问题的，即程序代码的不同部分是根据不同的平台来编译的。由于Java自身可以自动跨越不同的平台，因此这个功能对Java而言是没有必要的。

然而，条件编译还有其他一些有价值的用途。调试就是一个很常见的用途。调试功能在开发过程中是开启的，而在发布的产品中是禁用的。可以通过修改被导入的**package**的方法来实现这一目的，修改的方法是将你程序中用到的代码从调试版改为发布版。这一技术可以适用于任何种类的条件代码。

练习3：(2) 创建两个包：**debug**和**debugoff**，它们都包含一个相同的类，该类有一个**debug()**方法。第一个版本显示发送给控制台的**String**参数，而第二个版本什么也不做。使用静态**import**语句将该类导入到一个测试程序中，并示范条件编译效果。

6.1.5 对使用包的忠告

务必记住，无论何时创建包，都已经在给定包的名称的时候隐含地指定了目录结构。这个包必须位于其名称所指定的目录之中，而该目录必须是在以CLASSPATH开始的目录中可以查询

到的。最初使用关键字**package**，可能会有一点不顺，因为除非遵守“包的名称对应目录路径”的规则，否则将会收到许多出乎意料的运行时信息，告知无法找到特定的类，哪怕是这个类就位于同一个目录之中。如果你收到类似信息，就用注释掉**package**语句的方法试一下，如果这样程序就能运行的话，你就可以知道问题出在哪里了。

220

注意，编译过的代码通常放置在与源代码的不同目录中，但是必须保证JVN使用CLASSPATH可以找到该路径。

6.2 Java访问权限修饰词

public、**protected**和 **private**这几个Java访问权限修饰词在使用时，是置于类中每个成员的定义之前的–无论它是一个域还是一个方法。每个访问权限修饰词仅控制它所修饰的特定定义的访问权。

如果不提供任何访问权限修饰词，则意味着它是“包访问权限”。因此，无论如何，所有事物都具有某种形式的访问权限控制。在以下几节中，读者将学习各种类型的访问权限。

6.2.1 包访问权限

本章之前的所有示例都没有使用任何访问权限修饰词。默认访问权限没有任何关键字，但通常是指包访问权限（有时也表示成为friendly）。这就意味着当前的包中的所有其他类对那个成员都有访问权限，但对于这个包之外的所有类，这个成员却是**private**。由于一个编译单元（即一个文件），只能隶属于一个包，所以经由包访问权限，处于同一个编译单元中的所有类彼此之间都是自动可访问的。

包访问权限允许将包内所有相关的类组合起来，以使它们彼此之间可以轻松地相互作用。当把类组织起来放进一个包内之时，也就给它们的包访问权限的成员赋予了相互访问的权限，你“拥有”了该包内的程序代码。“只有你拥有的程序代码才可以访问你所拥有的其他程序代码”，这是合理的。应该说，包访问权限为把类群聚在一个包中的做法提供了意义和理由。在许多语言中，在文件内组织定义的方式是任意的，但在Java中，则要强制你以一种合理的方式对它们加以组织。另外，你可能还想要排除这样的类–它们不应该访问在当前包中所定义的类。

类控制着哪些代码有权访问自己的成员。其他包内的类不能刚一上来就说：“嗨，我是Bob的朋友。”并且还想看到Bob的**protected**、包访问权限和**private**成员。取得对某成员的访问权的唯一途径是：

221

1. 使该成员成为**public**。于是，无论是谁，无论在哪里，都可以访问该成员。
2. 通过不加访问权限修饰词并将其他类放置于同一个包内的方式给成员赋予包访问权。于是包内的其他类也就可以访问该成员了。
3. 在第7章将会介绍继承技术，届时读者将会看到继承而来的类既可以访问**public**成员也可以访问**protected**成员（但访问**private**成员却不行）。只有在两个类都处于同一个包内时，它才可以访问包访问权限的成员。但现在不必担心继承和**protected**。
4. 提供访问器（accessor）和变异器（mutator）方法（也称作get/set方法），以读取和改变数值。正如将在第22章中看到的，对OOP而言，这是最优雅的方式，而且这也是JavaBeans的基本原理。

6.2.2 public：接口访问权限

使用关键字**public**，就意味着**public**之后紧跟着的成员声明自己对每个人都是可用的，尤其是使用类库的客户程序员更是如此。假设定义了一个包含下面编译单元的**dessert**包：

```
//: access/dessert/Cookie.java
// Creates a library.
package access.dessert;

public class Cookie {
  public Cookie() {
   System.out.println("Cookie constructor");
  }
  void bite() { System.out.println("bite"); }
} ///:~
```

记住，**Cookie.java**文件必须位于名为**dessert**的子目录之中，该子目录在**access**（意指本书第6章）下，而c05则必须位于CLASSPATH指定的众多路径的其中之一的下边。不要错误地认为Java总是将当前目录视作是查找行为的起点之一。如果你的CLASSPATH之中缺少一个“.”作为路径之一的话，Java就不会查找那里。 222

现在如果创建了一个使用Cookie的程序：

```
//: access/Dinner.java
// Uses the library.
import access.dessert.*;

public class Dinner {
  public static void main(String[] args) {
    Cookie x = new Cookie();
    //! x.bite(); // Can't access
  }
} /* Output:
Cookie constructor
*///:~
```

就可以创建一个Cookie对象，因为它的构造器是public而且类也是public的。（此后我们将会对public类的概念了解更多。）但是，由于bite()只向在dessert包中的类提供访问权，所以bite() 成员在Dinner.java之中是无法访问的，因此编译器也禁止你使用它。

默认包

令人吃惊的是，下面的程序代码虽然看起来破坏了上述规则，但它仍可以编译：

```
//: access/Cake.java
// Accesses a class in a separate compilation unit.

class Cake {
  public static void main(String[] args) {
    Pie x = new Pie();
    x.f();
  }
} /* Output:
Pie.f()
*///:~
```

在第二个处于相同目录的文件中：

```
//: access/Pie.java
// The other class.
class Pie {
  void f() { System.out.println("Pie.f()"); }
} ///:~
```

223

最初或许会认为这两个文件毫不相关，但**Cake**却可以创建一个**Pie**对象并调用它的**f()**方法！（记住，为了使文件可以被编译，在你的CLASSPATH之中一定要有“.”。）通常会认为**Pie**和**f()**享有包访问权限，因而是不可以为**Cake**所用的。它们的确享有包访问权限，但这只是部分正确的。**Cake.java**可以访问它们的原因是因为它们同处于相同的目录并且没有给自己设定任何包名称。Java将这样的文件自动看作是隶属于该目录的默认包之中，于是它们为该目录中所有其他的文

件都提供了包访问权限。

6.2.3 private: 你无法访问

关键字**private**的意思是，除了包含该成员的类之外，其他任何类都无法访问这个成员。由于处于同一个包内的其他类是不可以访问**private**成员的，因此这等于说是自己隔离了自己。从另一方面说，让许多人共同合作来创建一个包也是不大可能的，为此**private**就允许你随意改变该成员，而不必考虑这样做是否会影响到包内其他的类。

默认的包访问权限通常已经提供了充足的隐藏措施。请记住，使用类的客户端程序员是无法访问包访问权限成员的。这样做很好，因为默认访问权限是一种我们常用的权限，同时也是一种在忘记添加任何访问权限控制时能够自动得到的权限。因此，通常考虑的是，哪些成员是想要明确公开给客户端程序员使用的，从而将它们声明为**public**，而在最初，你可能不会认为自己经常会需要使用关键字**private**，因为没有它，照样可以工作。然而，事实很快就会证明，对**private**的使用是多么的重要，在多线程环境下更是如此（正如将在第21章中看到的）。

此处有一个使用**private**的示例。

```
//: access/IceCream.java
// Demonstrates "private" keyword.

class Sundae {
  private Sundae() {}
  static Sundae makeASundae() {
    return new Sundae();
  }
}

public class IceCream {
  public static void main(String[] args) {
    //! Sundae x = new Sundae();
    Sundae x = Sundae.makeASundae();
  }
} ///:~
```

224

这是一个说明**private**终有其用武之地的示例：可能想控制如何创建对象，并阻止别人直接访问某个特定的构造器（或全部构造器）。在上面的例子中，不能通过构造器来创建**Sundae**对象，而必须调用**makeASundae()**方法来达到此目的[⊖]。

任何可以肯定只是该类的一个“助手”方法的方法，都可以把它指定为**private**，以确保不会在包内的其他地方误用到它，于是也就防止了你会去改变或删除这个方法。将方法指定为**private**确保了你拥有这种选择权。

这对于类中的**private**域同样适用。除非必须公开底层实现细目（此种境况很少见），否则就应该将所有的域指定为**private**。然而，不能因为在类中某个对象的引用是**private**，就认为其他的对象无法拥有该对象的**public** 引用（参见本书的在线补充材料以了解有关别名机制的话题）。

6.2.4 protected：继承访问权限

要理解**protected**的访问权限，我们在内容上需要作一点跳跃。首先，在本书介绍继承（第7章）之前，读者并不需真正理解本节的内容。但为了内容的完整性，这里还是提供了一个简要介绍和使用**protected**的示例。

225

关键字**protected**处理的是继承的概念，通过继承可以利用一个现有类—我们将其称为基类，

⊖ 此例还有另一个效果：既然默认构造器是唯一定义的构造器，并且它是**private**的，那么它将阻碍对此类的继承（我们将在后面介绍这个问题）。

然后将新成员添加到该现有类中而不必碰该现有类。还可以改变该类的现有成员的行为。为了从现有类中继承，需要声明新类**extends**（扩展）了一个现有类，就像这样：

```
class Foo extends Bar {
```

类定义中的其他部分看起来都是一样的。

如果创建了一个新包，并自另一个包中继承类，那么唯一可以访问的成员就是源包的public成员。（当然，如果在同一个包内执行继承工作，就可以操纵所有的拥有包访问权限的成员。）有时，基类的创建者会希望有某个特定成员，把对它的访问权限赋予派生类而不是所有类。这就需要**protected**来完成这一工作。**protected**也提供包访问权限，也就是说，相同包内的其他类可以访问**protected**元素。

回顾一下先前的例子**Cookie.java**，就可以得知下面的类是不可以调用拥有包访问权限的成员**bite()**的：

```
//: access/ChocolateChip.java
// Can't use package-access member from another package.
import access.dessert.*;

public class ChocolateChip extends Cookie {
  public ChocolateChip() {
   System.out.println("ChocolateChip constructor");
  }
  public void chomp() {
    //! bite(); // Can't access bite
  }
  public static void main(String[] args) {
    ChocolateChip x = new ChocolateChip();
    x.chomp();
  }
} /* Output:
Cookie constructor
ChocolateChip constructor
*///:~
```

有关继承技术的一个很有趣的事情是，如果类**Cookie**中存在一个方法**bite()**的话，那么该方法同时也存在于任何一个从**Cookie**继承而来的类中。但是由于**bite()**有包访问权限而且它位于另一个包内，所以我们在这个包内是无法使用它的。当然，也可以把它指定为**public**，但是这样做所有的人就都有了访问权限，而且很可能这并不是你所希望的。如果我们将类**Cookie**像这样加以更改：

226

```
//: access/cookie2/Cookie.java
package access.cookie2;

public class Cookie {
  public Cookie() {
    System.out.println("Cookie constructor");
  }
  protected void bite() {
    System.out.println("bite");
  }
} ///:~
```

现在**bite()**对于所有继承自**Cookie**的类而言，也是可以使用的。

```
//: access/ChocolateChip2.java
import access.cookie2.*;

public class ChocolateChip2 extends Cookie {
  public ChocolateChip2() {
   System.out.println("ChocolateChip2 constructor");
  }
```

```
  public void chomp() { bite(); } // Protected method
  public static void main(String[] args) {
    ChocolateChip2 x = new ChocolateChip2();
    x.chomp();
  }
} /* Output:
Cookie constructor
ChocolateChip2 constructor
bite
*///:~
```

注意，尽管**bite()**也具有包访问权限，但是它仍旧不是**public**的。

练习4：(2) 展示**protected**方法具有包访问权限，但不是**public**。

练习5：(2) 创建一个带有**public, private, protected**和包访问权限域以及方法成员的类。创建该类的一个对象，看看在你试图调用所有类成员时，会得到什么类型的编译信息。请注意，处于同一个目录中的所有类都是默认包的一部分。

227

练习6：(1) 创建一个带有**protected**数据的类。运用在第一个类中处理**protected**数据的方法在相同的文件中创建第二个类。

6.3 接口和实现

访问权限的控制常被称为是*具体实现的隐藏*。把数据和方法包装进类中，以及具体实现的隐藏，常共同被称作是*封装*[⊖]。其结果是一个同时带有特征和行为的数据类型。

出于两个很重要的原因，访问权限控制将权限的边界划在了数据类型的内部。第一个原因是要设定客户端程序员可以使用和不可以使用的界限。可以在结构中建立自己的内部机制，而不必担心客户端程序员会偶然地将内部机制当作是他们可以使用的接口的一部分。

这个原因直接引出了第二个原因，即将接口和具体实现进行分离。如果结构是用于一组程序之中，而客户端程序员除了可以向接口发送信息之外什么也不可以做的话，那么就可以随意更改所有不是**public**的东西（例如有包访问权限、**protected** 和 **private**的成员），而不会破坏客户端代码。

为了清楚起见，可能会采用一种将**public**成员置于开头，后面跟着**protected**、包访问权限和**private**成员的创建类的形式。这样做的好处是类的使用者可以从头读起，首先阅读对他们而言最为重要的部分（即**public**成员，因为可以从文件外部调用它们），等到遇见作为内部实现细节的非**public**成员时停止阅读：

```
//: access/OrganizedByAccess.java

public class OrganizedByAccess {
  public void pub1() { /* ... */ }
  public void pub2() { /* ... */ }
  public void pub3() { /* ... */ }
  private void priv1() { /* ... */ }
  private void priv2() { /* ... */ }
  private void priv3() { /* ... */ }
  private int i;
  // ...
} ///:~
```

228

这样做仅能使程序阅读起来稍微容易一些，因为接口和具体实现仍旧混在一起。也就是说，仍能看到源代码—实现部分，因为它就在类中。另外，javadoc所提供的*注释文档*功能降低了程序代码的可读性对客户端程序员的重要性。将接口展现给某个类的使用者实际上是类浏览器的

⊖ 然而，人们经常只单独将具体实现的隐藏称作封装。

任务。类浏览器是一种以非常有用的方式来查阅所有可用的类，并告诉你用它们可以做些什么（也就是显示出可用成员）的工具。在Java中，用Web浏览器浏览JDK文档可以得到使用类浏览器的相同效果。

6.4 类的访问权限

在Java中，访问权限修饰词也可以用于确定库中的哪些类对于该库的使用者是可用的。如果希望某个类可以为某个客户端程序员所用，就可以通过把关键字**public**作用于整个类的定义来达到目的。这样做甚至可以控制客户端程序员是否能创建一个该类的对象。

为了控制某个类的访问权限，修饰词必须出现于关键字**class**之前。因此可以像下面这样声明：

```
public class Widget {
```

现在如果库的名字是**access**，那么任何客户端程序员都可以通过下面的声明访问**Widget**：

```
import access.Widget;
```

或

```
import access.*;
```

然而，这里还有一些额外的限制：

229

1. 每个编译单元（文件）都只能有一个**public**类。这表示，每个编译单元都有单一的公共接口，用**public**类来表现。该接口可以按要求包含众多的支持包访问权限的类。如果在某个编译单元内有一个以上的**public**类，编译器就会给出出错信息。

2. **public**类的名称必须完全与含有该编译单元的文件名相匹配，包括大小写。所以对于**Widget**而言，文件的名称必须是**Widget.java**，而不是**widget.java**或**WIDGET.java**。如果不匹配，同样将得到编译时错误。

3. 虽然不是很常用，但编译单元内完全不带**public**类也是可能的。在这种情况下，可以随意对文件命名。（尽管随意命名会使得人们在阅读和维护代码时产生混淆。）

如果获取了一个在**access**内部的类，用来完成**Widget**或是其他在**access**中的**public**类所要执行的任务，将会出现什么样的情况呢？你不想自找麻烦去为客户端程序员创建说明文档，而且你认为不久可能会想要完全改变原有方案并将旧版本一起删除，代之以一种不同的版本。为了保留此灵活性，需要确保客户端程序员不会依赖于隐藏在**access**之中的任何特定实现细节。为了达到这一点，只需将关键字**public**从类中拿掉，这个类就拥有了包访问权限。（该类只可以用于该包之中。）

练习7：(1) 根据描述**access**和**Widget**的代码片段创建类库。在某个不属于**access**类库的类中创建一个**Widget**实例。

在创建一个包访问权限的类时，仍旧是在将该类的域声明为**private**时才有意义—应尽可能地总是将域指定为私有的，但是通常来说，将与类（包访问权限）相同的访问权限赋予方法也是很合理的。既然一个有包访问权限的类通常只能被用于包内，那么如果对你有强制要求，在此种情况下，编译器会告诉你，你只需要将这样的类的方法设定为**public**就可以了。

230

请注意，类既不可以是**private**的（这样会使得除该类之外，其他任何类都不可以访问它），也不可以是**protected**的[⊖]。所以对于类的访问权限，仅有两个选择：包访问权限或**public**。如果不希望其他任何人对该类拥有访问权限，可以把所有的构造器都指定为**private**，从而阻止任何人创建该类的对象，但是有一个例外，就是你在该类的**static**成员内部可以创建。下面是一个示例：

⊖ 事实上，一个内部类可以是**private**或是**protected**的，但那是一个特例。这将在第10章中介绍到。

```
//: access/Lunch.java
// Demonstrates class access specifiers. Make a class
// effectively private with private constructors:

class Soup1 {
  private Soup1() {}
  // (1) Allow creation via static method:
  public static Soup1 makeSoup() {
    return new Soup1();
  }
}

class Soup2 {
  private Soup2() {}
  // (2) Create a static object and return a reference
  // upon request.(The "Singleton" pattern):
  private static Soup2 ps1 = new Soup2();
  public static Soup2 access() {
    return ps1;
  }
  public void f() {}
}

// Only one public class allowed per file:
public class Lunch {
  void testPrivate() {
    // Can't do this! Private constructor:
    //! Soup1 soup = new Soup1();
  }
  void testStatic() {
    Soup1 soup = Soup1.makeSoup();
  }
  void testSingleton() {
    Soup2.access().f();
  }
} ///:~
```

231

到目前为止，绝大多数方法均返回void或基本类型，所以定义

```
public static Soup1 makeSoup() {
  return new Soup1();
}
```

初看起来可能有点令人迷惑不解。方法名称（**makeSoup**）前面的词**Soup1**告知了该方法返回的东西。本书到目前为止，这里经常是**void**，意思是它不返回任何东西。但是也可以返回一个对象引用，示例中就是这种情况。这个方法返回了一个对**Soup1**类的对象的引用。

Soup1类和**Soup2**类展示了如何通过将所有的构造器指定为**private**来阻止直接创建某个类的实例。请一定要牢记，如果没有明确地至少创建一个构造器的话，就会帮你创建一个默认构造器（不带有任何参数的构造器）。如果我们自己编写了默认的构造器，那么就不会自动创建它了。如果把该构造器指定为**private**，那么就谁也无法创建该类的对象了。但是现在别人该怎样使用这个类呢？上面的例子就给出了两种选择：在**Soup1**中，创建一个**static**方法，它创建一个新的**Soup1**对象并返回一个对它的引用。如果想要在返回引用之前在**Soup1**上做一些额外的工作，或是如果想要记录到底创建了多少个**Soup1**对象（可能要限制其数量），这种做法将会是大有裨益的。

Soup2用到了所谓的设计模式，该模式在www.MindView.net网站《Thinking in Patterns (with Java)》一书中有所介绍。这种特定的模式被称为singleton(单例)，这是因为你始终只能创建它的一个对象。**Soup2**类的对象是作为**Soup2**的一个**static private** 成员而创建的，所以有且仅有一个，而且除非是通过**public**方法**access()**，否则是无法访问到它的。

正如前面所提到的，如果没能为类访问权限指定一个访问修饰符，它就会默认得到包访问权限。这就意味着该类的对象可以由包内任何其他类来创建，但在包外则是不行的。（一定要记住，相同目录下的所有不具有明确**package**声明的文件，都被视作是该目录下默认包的一部分。）然而，如果该类的某个**static**成员是**public**的话，则客户端程序员仍旧可以调用该**static**成员，尽管他们并不能生成该类的对象。 232

练习8：(4) 效仿示例**Lunch.java**的形式，创建一个名为**ConnectionManager**的类，该类管理一个元素为**Connection**对象的固定数组。客户端程序员不能直接创建**Connection**对象，而只能通过**ConnectionManager**中的某个**static**方法来获取它们。当**ConnectionManager**之中不再有对象时，它会返回**null**引用。在**main()**之中检测这些类。

练习9：(2) 在**access/local**目录下编写以下文件（假定**access/local**目录在你的CLASSPATH中）：

```
// access/local/PackagedClass.java
package access.local;

class PackagedClass {
  public PackagedClass() {
    System.out.println("Creating a packaged class");
  }
}
```

然后在**accesss/local**之外的另一个目录中创建下列文件：

```
// access/foreign/Foreign.java
package access.foreign;
import access.local.*;

public class Foreign {
   public static void main(String[] args) {
      PackagedClass pc = new PackagedClass();
   }
}
```

解释一下为什么编译器会产生错误。如果将**Foreign**类置于**access.local**包之中的话，会有所改变吗？

6.5 总结

无论是在什么样的关系之中，设立一些为各成员所遵守的界限始终是很重要的。当创建了一个类库，也就与该类库的用户建立了某种关系，这些用户就是客户端程序员，他们是另外一些程序员，他们将你的类库聚合成为一个应用程序，或是运用你的类库来创建一个更大的类库。 233

如果不制定规则，客户端程序员就可以对类的所有成员随心而为，即使你可能并不希望他们直接复制其中的一些成员。在这种情况下，所有事物都是公开的。

本章讨论了类是如何被构建成类库的：首先，介绍了一组类是如何被打包到一个类库中的；其次，类是如何控制对其成员的访问的。

据估计，用C语言开发项目，在50千行至100千行代码之间就会出现问题。这是因为C语言仅有单一的“名字空间”，并且名称开始发生冲突，引发额外的管理开销。而对于Java，关键字**package**、包的命名模式和关键字**import**，可以使你对名称进行完全的控制，因此名称冲突的问题是很容易避免的。

控制对成员的访问权限有两个原因。第一是为了使用户不要碰触那些他们不该碰触的部分，这些部分对于类内部的操作是必要的，但是它并不属于客户端程序员所需接口的一部分。因此，将方法和域指定成**private**，对客户端程序员而言是一种服务。因为这样他们可以很清楚地看到

什么对他们重要，什么是他们可以忽略的。这样简化了他们对类的理解。

第二个原因，也是最重要的原因，是为了让类库设计者可以更改类的内部工作方式，而不必担心这样会对客户端程序员产生重大的影响。例如，最初可能会以某种方式创建一个类，然后发现如果更改程序结构，可以大大提高运行速度。如果接口和实现可以被明确地隔离和加以保护，那么就可以实现这一目的，而不必强制客户端程序员重新编写代码。访问权限控制可以确保不会有任何客户端程序员依赖于某个类的底层实现的任何部分。

当具备了改变底层实施细节的能力时，不仅可以随意地改善设计，还可能会随意地犯错误，同时也就有了犯错的可能性。无论如何细心地计划并设计，都有可能犯错。当了解到你所犯错误是相对安全的时候，就可以更加放心地进行实验，也就可以更快地学会，更快地完成项目。

234

类的公共接口是用户真正能够看到的，所以这一部分是在分析和设计的过程中决定该类是否正确的最重要的部分。尽管如此，你仍然有进行改变的空间。如果在最初无法创建出正确的接口，那么只要不删除任何客户端程序员在他们的程序中已经用到的东西，就可以在以后添加更多的方法。

注意，访问权限控制专注于类库创建者和该类库的外部使用者之间的关系，这种关系也是一种通信方式。然而，在许多情况下事情并非如此。例如，你自己编写了所有的代码，或者你在一个组员聚集在一起的项目组中工作，所有的东西都放在同一个包中。这些情况是另外一种不同的通信方式，因此严格地遵循访问权限规则并不一定是最佳选择，默认（包）访问权限也许只是可行而已。

235 ~ 236

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net购买此文档。